

Hash

字符串哈希

定义

我们定义一个把字符串映射到整数的函数，这个 称为是 Hash 函数。

我们希望这个函数 可以方便地帮我们判断两个字符串是否相等。

Hash 的思想

Hash 的核心思想在于，将输入映射到一个值域较小、可以方便比较的范围。

► Warning

性质

具体来说，哈希函数最重要的性质可以概括为下面两条：

1. 在 Hash 函数值不一样的时候，两个字符串一定不一样；
2. 在 Hash 函数值一样的时候，两个字符串不一定一样（但有大概率一样，且我们当然希望它们总是一样的）。

我们将 Hash 函数值一样但原字符串不一样的现象称为哈希碰撞。

解释

我们需要关注的是什么呢？

时间复杂度和 Hash 的准确率。

通常我们采用的是多项式 Hash 的方法，对于一个长度为 n 的字符串 s 来说，我们可以这样定义多项式 Hash 函数： $h(s) = s_0 \cdot x^0 + s_1 \cdot x^1 + \dots + s_{n-1} \cdot x^{n-1}$ 。例如，对于字符串 "abc"，其哈希函数值为 $h("abc") = 'a' \cdot x^0 + 'b' \cdot x^1 + 'c' \cdot x^2$ 。

特别要说明的是，也有很多人使用的是另一种 Hash 函数的定义，即 $h(s) = s_0 \cdot x^{n-1} + s_1 \cdot x^{n-2} + \dots + s_{n-1} \cdot x^0$ ，这种定义下，同样的字符串 s 的哈希值就变为了 $h(s) = 'c' \cdot x^2 + 'b' \cdot x^1 + 'a' \cdot x^0$ 。

显然，上面这两种哈希函数的定义函数都是可行的，但二者在之后会讲到的计算子串哈希值时所用的计算式是不同的，因此千万注意 **不要弄混了这两种不同的 Hash 方式**。

由于前者的 Hash 定义计算更简便、使用人数更多、且可以类比为一个 进制数来帮助理解，所以本文下面所将要讨论的都是使用 $h(s) = s_0 \cdot x^0 + s_1 \cdot x^1 + \dots + s_{n-1} \cdot x^{n-1}$ 来定义的 Hash 函数。

还有，有时为了方便和扩大模数，我们在 C++ 中我们会使用 `unsigned long long` 来定义 Hash 函数的结果。由于 C++ 的特性，我们相当于把模数 定为 2^{64} ，也是一个不错的选择。

准确率会在后面讨论。

Hash 的错误率分析

Hash 冲突

Hash 冲突是指两个不同的字符串映射到相同的 Hash 值。

我们设 Hash 的取值空间（所有可能出现的字符串的数量）为 N ，计算次数（要计算的字符串数量）为 M 。

则 Hash 冲突的概率为：

► 证明

卡大模数 Hash

注意到这个公式：

为了卡掉 Hash，我们要满足一下条件：

1. 要大于模数。
2. 要尽量小。

举个例子：

若字符集为 **大小写字母和数字**，模数为 10^9 时：

所以对于这个范围，我们随机生成 10^9 个长度为 10 的字符串，它们 Hash 值相同的概率高达 10^{-9} 。

卡自然溢出 Hash

这种 Hash 由于模数太大，用上面的方法卡不了，所以我们需要另一种方法。

首先，这种 Hash 是形如 $H(s) = \sum_{i=0}^{n-1} s[i] \cdot 2^{bi}$ ，我们根据 b 来分类讨论。

b 为偶数

此时，其中 $2^{bi} = 1$ 。

容易发现若 $s_1 = a_1 a_2 \dots a_n$ ， $s_2 = b_1 a_2 \dots a_n$ 。

所以我们只要构造形如：

$aaa\dots a$

$baa\dots a$

且长度大于 $\frac{b}{2}$ 的字符串就能冲突。

b 为奇数

定义 $reverse(s)$ 为把 s 中所有字符反转。

例：

即把 a 变成 b ，把 b 变成 a 。

再定义 h 为 a 的 Hash 值， h' 为 b 的 Hash 值。

不断构造。

h 和 h' 就是我们要的两个字符串。

► 推导

例题

▼ [例题：BZOJ 3097 Hash Killer I](#)

给定一个用 自然溢出 实现的 Hash，要求构造一个字符串来卡掉它。

▼ [例题：BZOJ 3097 Hash Killer II](#)

给定一个用 大模数 实现的 Hash，要求构造一个字符串来卡掉它。

▼ [例题：洛谷 U461211 字符串 Hash（数据加强）](#)

给定 n 个字符串，判断不同的字符串有多少个。

Hash 的改进

多值 Hash

看了上面这么多的卡法，当然也有解决办法。

多值 Hash，就是有多个 Hash 函数，每个 Hash 函数的模数不一样，这样就能解决 Hash 冲突的问题。

判断时只要有其中一个的 Hash 值不同，就认为两个字符串不同，若 Hash 值都相同，则认为两个字符串相同。

一般来说，双值 Hash 就够用了。

多次询问子串哈希

单次计算一个字符串的哈希值复杂度是 $O(n)$ ，其中 n 为串长，与暴力匹配没有区别，如果需要多次询问一个字符串的子串的哈希值，每次重新计算效率非常低下。

一般采取的方法是对整个字符串先预处理出每个前缀的哈希值，将哈希值看成一个 p 进制的数对 p 取模的结果，这样的话每次就能快速求出子串的哈希了：

令 h_i 表示 $s[1..i]$ 的前缀的哈希值，那么按照定义有

现在，我们想要用类似前缀和的方式快速求出 $s[l..r]$ 的哈希值为

对比观察上述两个式子，我们发现 $h_r - h_{l-1} \cdot p^{r-l+1}$ 成立（可以手动代入验证一下），因此我们用这个式子就可以快速得到子串的哈希值。其中 p^i 可以预处理出来然后 $h_r - h_{l-1} \cdot p^{r-l+1}$ 的回答每次询问（当然也可以快速幂 的回答每次询问）。

实现

模数 Hash：

注：效率较低，实际使用中不推荐。



C++Python

```
1 using std::string;
2
3 constexpr int M = 1e9 + 7;
4 constexpr int B = 233;
5
6 using ll = long long;
7
8 int get_hash(const string& s) {
9     int res = 0;
10    for (int i = 0; i < s.size(); ++i) {
11        res = ((ll)res * B + s[i]) % M;
12    }
13    return res;
14 }
15
16 bool cmp(const string& s, const string& t) {
17     return get_hash(s) == get_hash(t);
18 }
```

```
1 M = int(1e9 + 7)
2 B = 233
3
4
5 def get_hash(s):
6     res = 0
7     for char in s:
8         res = (res * B + ord(char)) % M
9     return res
10
11
12 def cmp(s, t):
13     return get_hash(s) == get_hash(t)
```

双值 Hash:



C++Python

```

1 using ull = unsigned long long;
2 ull base = 131;
3 ull mod1 = 212370440130137957, mod2 = 1e9 + 7;
4
5 ull get_hash1(std::string s) {
6     int len = s.size();
7     ull ans = 0;
8     for (int i = 0; i < len; i++) ans = (ans * base + (ull)s[i]) % mod1;
9     return ans;
10 }
11
12 ull get_hash2(std::string s) {
13     int len = s.size();
14     ull ans = 0;
15     for (int i = 0; i < len; i++) ans = (ans * base + (ull)s[i]) % mod2;
16     return ans;
17 }
18
19 bool cmp(const std::string s, const std::string t) {
20     bool f1 = get_hash1(s) != get_hash1(t);
21     bool f2 = get_hash2(s) != get_hash2(t);
22     return f1 || f2;
23 }

```

```

1 def get_hash1(s: str) -> int:
2     base = 131
3     mod1 = 212370440130137957
4     ans = 0
5     for char in s:
6         ans = (ans * base + ord(char)) % mod1
7     return ans
8
9
10 def get_hash2(s: str) -> int:
11     base = 131
12     mod2 = 1000000007
13     ans = 0
14     for char in s:
15         ans = (ans * base + ord(char)) % mod2
16     return ans
17
18
19 def cmp(s: str, t: str) -> bool:
20     f1 = get_hash1(s) != get_hash1(t)
21     f2 = get_hash2(s) != get_hash2(t)
22     return f1 or f2

```

...

Hash 的应用

字符串匹配

求出模式串的哈希值后，求出文本串每个长度为模式串长度的子串的哈希值，分别与模式串的哈希值比较即可。

允许 次失配的字符串匹配

问题：给定长为 n 的源串 S ，以及长度为 m 的模式串 T ，要求查找源串中有多少子串与模式串匹配。与 T 匹配，当且仅当与 T 长度相同，且最多有 k 个位置字符不同。其中， $k \leq m$ 。

这道题无法使用 KMP 解决，但是可以通过哈希 + 二分来解决。

枚举所有可能匹配的子串，假设现在枚举的子串为 $s[l:r]$ ，通过哈希 + 二分可以快速找到与 s 第一个不同的位置。之后将与 s 在这个失配位置及之前的部分删除掉，继续查找下一个失配位置。这样的过程最多发生 $O(n)$ 次。

总的时间复杂度为 $O(n^2)$ 。

最长回文子串

二分答案，判断是否可行时枚举回文中心（对称轴），哈希判断两侧是否相等。需要分别预处理正着和倒着的哈希值。时间复杂度 $O(n^2)$ 。

这个问题可以使用 [manacher 算法](#) 在 $O(n)$ 的时间内解决。

通过哈希同样可以解决这个问题，具体方法就是记 len 表示以 i 作为结尾的最长回文的长度，那么答案就是 len 。考虑到，因此我们只需要暴力从 i 开始递减，直到找到第一个回文即可。记变量 l 表示当前枚举的，初始时为 i ，则 l 在每次 len 增大的时候都会增大，之后每次暴力循环都会减少，故暴力循环最多发生 $O(n)$ 次，总的时间复杂度为 $O(n^2)$ 。

最长公共子字符串

问题：给定 n 个总长不超过 m 的非空字符串，查找所有字符串的最长公共子字符串，如果有多个，任意输出其中一个。其中 $n \leq 100, m \leq 10^5$ 。

很显然如果存在长度为 k 的最长公共子字符串，那么 $k/2$ 的公共子字符串也必定存在。因此我们可以二分最长公共子字符串的长度。假设现在的长度为 k ， $check(k)$ 的逻辑为我们将所有字符串的长度为 k 的子串分别进行哈希，将哈希值放入 n 个哈希表中存储。之后求交集即可。

时间复杂度为 $O(nm \log m)$ 。

确定字符串中不同子字符串的数量

问题：给定长为 n 的字符串，仅由小写英文字母组成，查找该字符串中不同子串的数量。

为了解决这个问题，我们遍历了所有长度为 k 的子串。对于每个长度为 k 的子串，我们将其 Hash 值乘以相同的 k 的幂次方，并存入一个数组中。数组中不同元素的数量等于字符串中长度不同的子串的数量，并此数字将添加到最终答案中。

为了方便起见，我们将使用 10^9+7 作为 Hash 的前缀字符，并定义 $h[k]$ 为长度为 k 的子串的 Hash 值。

► 参考代码

例题

▼ [CF1200E Compress Words](#)

给你若干个字符串，答案串初始为空。第 i 步将第 i 个字符串加到答案串的后面，但是尽量地去掉重复部分（即去掉一个最长的、是原答案串的后缀、也是第 i 个串的前缀的字符串），求最后得到的字符串。

字符串个数不超过 10^5 ，总长不超过 10^6 。

► 题解

► 参考代码

本页面部分内容译自博文 [строковый хеш](#) 与其英文翻译版 [String Hashing](#)。其中俄文版版权协议为 Public Domain + Leave a Link；英文版版权协议为 CC-BY-SA 4.0。

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [1292224662](#), [algoriiiiithm](#), [Chrogeek](#), [Enter-tainer](#), [GldHkkowo](#), [Haohu Shen](#), [HeRaNO](#), [ImpleLee](#), [lr1d](#), [kenlig](#), [ksyx](#), [Marcythm](#), [Menci](#), [mgt](#), [Molmin](#), [ouuan](#), [runnableAir](#), [shawlleyw](#), [ShuYuMo2003](#), [sshwy](#), [szdytom](#), [taodaling](#), [tfia](#), [Tiphereth-A](#), [wangchong](#), [wendajiang](#), [Xeonacid](#), [xglight](#), [xiangmy21](#), [Yanjun-Zhao](#), [zyouxam](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用